

# Tables in R - Data frames and Tibbles

Day 1 - Introduction to Data Analysis with R

Selina Baldauf

Freie Universität Berlin - Theoretical Ecology

October 7, 2026

# Data frames

The built-in data structure for tables in R is a **data frame**.

Vectors in R can't represent a data table where values are connected via rows

Data frames are one of the **biggest and most important ideas** in R, and one of the things that make R different from other programming languages.

(H. Wickham, [Advanced R](#))

| city_name        | population_size | city_area |
|------------------|-----------------|-----------|
| Istanbul         | 15100000        | 2576      |
| Moscow           | 12500000        | 2561      |
| London           | 9000000         | 1572      |
| Saint Petersburg | 5400000         | 1439      |
| Berlin           | 3800000         | 891       |
| Madrid           | 3200000         | 604       |
| Kyiv             | 3000000         | 839       |
| Rome             | 2800000         | 1285      |
| Bucharest        | 2200000         | 228       |
| Paris            | 2100000         | 105       |

# Properties of a data frame

A data frame is a **named list of vectors** of the same length.

- every **column is a vector**
- columns have a **header**
- within one column, all values are of the **same data type**
- every column has the same length

The diagram shows a table representing a data frame. The header row is highlighted with a yellow border and contains three columns: 'cities', 'population', and 'area\_km2'. Above the 'cities' column, the word 'character' has a pink arrow pointing down to it. Above the 'population' and 'area\_km2' columns, the word 'numeric' has two pink arrows pointing down to each respectively. To the left of the table, a long green arrow points downwards, indicating that all rows have the same length. The table contains 10 rows of data, with the first row being the header and the following 9 rows containing city names, their populations, and their areas in square kilometers.

| cities           | population | area_km2 |
|------------------|------------|----------|
| Istanbul         | 15100000   | 2576     |
| Moscow           | 12500000   | 2561     |
| London           | 9000000    | 1572     |
| Saint Petersburg | 5400000    | 1439     |
| Berlin           | 3800000    | 891      |
| Madrid           | 3200000    | 604      |
| Kyiv             | 3000000    | 839      |
| Rome             | 2800000    | 1285     |
| Bucharest        | 2200000    | 228      |
| Paris            | 2100000    | 105      |

# Creating data frames

Data frames are created with the function `data.frame()`:

```
cities <- c(
  "Istanbul", "Moscow", "London",
  "Saint Petersburg", "Berlin", "Madrid",
  "Kyiv", "Rome", "Bucharest", "Paris")

population <- c(
  15.1e6, 12.5e6, 9e6, 5.4e6, 3.8e6,
  3.2e6, 3e6, 2.8e6, 2.2e6, 2.1e6)

area_km2 <- c(2576, 2561, 1572, 1439,
  891, 604, 839, 1285, 228, 105)

cities_dataframe <- data.frame(
  city_name = cities,
  population_size = population,
  city_area = area_km2
)
```

```
#>           city_name population_size city_area
#> 1           Istanbul      15100000      2576
#> 2             Moscow      12500000      2561
#> 3             London       9000000      1572
#> 4 Saint Petersburg       5400000      1439
#> 5              Berlin       3800000       891
#> 6              Madrid       3200000       604
#> 7               Kyiv       3000000       839
#> 8               Rome       2800000      1285
#> 9          Bucharest       2200000       228
#> 10             Paris       2100000       105
```

# Tibbles

Tibbles are

a **modern reimagining of the data frame**. Tibbles are designed to be (as much as possible) **drop-in replacements** for data frames.

(Wickham, [Advanced R](#))

Have a look at [this book chapter](#) for a full list of the differences between data frames and tibbles and the advantages of using tibbles.

- Tibbles have the same basic properties as data frames
- Everything that you can do with data frames, you can do with tibbles
- Tibbles are part of the **tibble** package

# Packages

Base R functions and data types: built into R

```
mean()          # calculate mean  
summary()       # show summary of a table  
data.frame()    # create a data frame
```

Additional **packages** extend R with new functions and data types

```
# install once (in the console, not in the script)  
install.packages("packageName")  
  
# load every time R restarts (at the top of your script)  
library(packageName)
```

# Tibbles

Create a tibble using the `tibble()` function:

```
# install the tibble package once
# install.packages("tibble")
library(tibble)

cities_tbl <- tibble(
  city_name = cities,
  population_size = population,
  city_area = area_km2
)
```

```
#> Error in `tibble()`:
#> ! could not find function "tibble"
#> Error:
#> ! object 'cities_tbl' not found
```

# Exploring tibbles

How many rows?

```
nrow(cities_tbl)
#> Error:
#> ! object 'cities_tbl' not found
```

How many columns?

```
ncol(cities_tbl)
#> Error:
#> ! object 'cities_tbl' not found
```

What are the column headers?

```
names(cities_tbl)
#> Error:
#> ! object 'cities_tbl' not found
```

# Exploring tibbles

Look at the entire table in a separate window with `view()`:

```
view(cities_tbl)
```

# Exploring tibbles

Get a quick summary of all columns:

```
summary(cities_tbl)
#> Error:
#> ! object 'cities_tbl' not found
```

- Very useful for checking if everything is ok with your research data

# Indexing tibbles

Indexing tibbles works similar to indexing vectors but with 2 dimensions instead of 1:

`tibble [ row_index, col_index or col_name ]`

- Missing `row_index` or `col_index` means *all rows* or *all columns* respectively.
- Indexing a tibble using `[]` always returns another tibble.

# Indexing tibbles

```
# First row and first column  
cities_tbl[1, 1]  
#> Error:  
#> ! object 'cities_tbl' not found
```

This is the same as

```
cities_tbl[1, "city_name"]
```

# Indexing tibbles: rows

```
# rows 1 & 5, all columns:  
cities_tbl[c(1, 5), ]  
#> Error:  
#> ! object 'cities_tbl' not found
```

# Indexing tibbles: columns

```
# All rows, first 2 columns  
cities_tbl[ ,1:2] # same as cities_tbl[ , c(1, 2)]  
# same as  
cities_tbl[ ,c("city_name", "population_size")]
```

#> Error:

#> ! object 'cities\_tbl' not found

# Indexing tibbles: columns

Indexing columns by name is usually preferred to indexing by position

```
cities_tbl[ ,1:2] # okay  
cities_tbl[ ,c("city_name", "population_size")] # better
```

## Why?

- Code is much easier to read
- Code is more robust against
  - changes in column order
  - mistakes in the code (e.g. typos)

```
cities_tbl[ ,c(1,3)] # 3 instead of 2 -> wrong but no error  
cities_tbl[ ,c("city_name", "popluation_size")] # typo -> wrong and error
```

### General rule

Good code produces errors when something unintended or wrong happens

# Tibbles: Select columns with \$

Select an entire column from a tibble using `$` (this returns a vector instead of a tibble):

```
cities_tbl$city_name  
#> Error:  
#> ! object 'cities_tbl' not found
```

# Adding new columns

New columns can be added as vectors using the **\$** operator. The vectors need to have the same length as the tibble has rows.

```
# add a country column
cities_tbl$country <- c(
  "Turkey", "Russia", "UK", "Russia", "Germany", "Spain",
  "Ukraine", "Italy", "Romania", "France"
)
#> Error:
#> ! object 'cities_tbl' not found
```

```
#> Error:
#> ! object 'cities_tbl' not found
```

# Summary I

- Install and load a package before using it:

```
# install the tibble package once
# install.packages("tibble")
library(tibble)
```

## Creating tibbles and data frames

```
# data frame
data.frame(
  a = 1:3,
  b = c("a", "b", "c"),
  c = c(TRUE, FALSE, FALSE)
)
# tibble
tibble(
  a = 1:3,
  b = c("a", "b", "c"),
  c = c(TRUE, FALSE, FALSE)
)
```

# Summary II

## Looking at tibble structure

```
# number of rows
nrow(tbl)
# number of columns
ncol(tbl)
# column headers
names(tbl)
# look at the data in a new window
view(tbl)
# summary of values from each column
summary(tbl)
```

# Summary III

## Indexing tibbles and selecting columns

Return result as tibble:

```
# rows and columns by position
tbl[1:3, c(1, 3)]
tbl[1:3, ] # all columns
tbl[, 3] # column 3, all rows
tbl[3] # same as above

# columns by name
tbl[, c("colA", "colB")]
tbl[c("colA", "colB")]
```

Return result as vector:

```
tbl$colA # select colA
```

# Now you

Task (15 min)

Tibbles

Find the task description [here](#)